FREE LIVE SESSION · 30 MINUTES

DevOps for AI-ML Systems

Make models reproducible, pack them safely in containers, and let a pipeline refuse to ship a worse one.

PRESENTER Emmanuel N. **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of DevOps for AI-ML Systems, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

WHY THIS COURSE MATTERS

Ship the model the way great teams ship code

01

Same inputs, same model

Pin code, data, params and environment. Every model gets a receipt.

02

Pack it once

Run it anywhere: a safe container, nothing secret inside, not root.

03

No green, no ship

A robot and a quality gate on every push. Worse models never ship.

THE COURSE · MODULES 1–3

Three modules, one model you can trust in production

MODULE 1

Ship the Model Like Code

Pin the seed, write a receipt per model, catch a silent data change with git.

MODULE 2

Pack It Once

Containerize the model, find a secret hiding in your own image, and fix it.

MODULE 3

No Green, No Ship

Build a pipeline with a quality gate, watch it block a bad model, run it on GitHub.

01

MODULE 1 OF 3

Ship the Model Like Code

If you can't rebuild a model, you can't trust it. Same inputs, same model.

Same code. Same data. Different model.

91.2%

Run 1 · model sha256 2221968d…

91.2%

Run 2 · model sha256 ee18ccf8…

92.9%

Run 3 · model sha256 313255f0…

0

lines of code changed between runs

WARNING seed is not pinned. Without a fixed seed, every run starts somewhere new, so it ends somewhere new.

A model is a function of four things

01 Code

Pin it with a git commit.

02 Data

Pin it with a fingerprint (sha256), or version it in git when small.

03 Params

Seed, epochs, learning rate. In params.json, committed to git.

04 Environment

Python and library versions. Pinned versions now; a container in Module 2.

Software: binary = f(code). Model: model = f(code, data, params, environment). Pin all four.

None of these change the code, so review never sees them

01

The unpinned seed

seed: null means a new random start every run. Pin it: "seed": 42.

02

The edited data

Someone fixes a few labels in the CSV. Accuracy moves; nobody knows why.

03

The upgraded library

pip install grabs a newer version on the server than on your laptop.

Pin the seed. Train twice. Commit.

TERMINAL

```
$ sed -i 's/"seed": null/"seed": 42/' params.json &&
git diff
$ python3 src/train.py && python3 src/train.py
$ git commit -am "pin seed for reproducible training"
$ python3 src/train.py && cat model/card.json
```

`sed -i 's/old/new/'`
search and replace inside the file

`git diff`
one line changed: - null, + 42

`train twice`
accuracy 92.9%, model sha256 94fe58ef… both times

`model/card.json`
the receipt: commit, data hash, params, accuracy

YOU SHOULD SEE "git_commit": "a20e20a", "data_sha256": "f4aa07aab998dfe2", "seed_used": 42, "accuracy": 0.9292

Catch a teammate's silent change in one minute

1

Run the teammate

bash .lab/teammate.sh quietly relabels some tickets. No code changes.

2

Ask git

git status --short: M data/tickets.csv. git diff --stat: 192 lines.

3

Read the receipt

Accuracy drops to 81.7%, and data_sha256 has a new fingerprint.

4

Roll back and tag

git restore data/tickets.csv, retrain: 94fe58… is back. Tag model-v1.



If you can't rebuild it, you can't trust it.

Pin the seed. Fingerprint the data. Params in a file. A receipt for every model. Git is your time machine.

02

MODULE 2 OF 3

Pack It Once

Your model works on your laptop. That's not the goal. Pack it once. Run it anywhere.

Stop making the server match you

WITHOUT A CONTAINER

Laptop 3.13 ✓ · Server 3.8 ×

Same code, same data, same seed. Three environments can give three different models.

VS

WITH A CONTAINER

The box brings what it needs

Python 3.12, src/ and model/ packed together. Like a lunchbox: packed at home, eaten anywhere.

Image = the packed lunchbox, built once, read-only. Container = lunch being eaten: a running copy you can throw away.

The key lines of a good model image

TERMINAL

```
FROM python:3.12-slim
COPY src/ src/
COPY model/ model/
USER appuser
```

FROM python:3.12-slim
the environment pin: the exact Python, every time

COPY src/ · model/
copy only what the model needs to run

USER appuser
don't run as root inside the box

HEALTHCHECK
Docker knows it's really answering: (healthy)

YOU SHOULD SEE Every line becomes a layer. Change one line and Docker rebuilds from there down.

COPY . . copies everything. Everything.

THE STARTER

COPY . .

Copies .env, .git, data/ and tests/ into the image.
Your token is now in every copy.

VS

THE FIX

COPY src/ src/
+ .dockerignore

Copy only what runs. Secrets arrive at run time with --env-file, never baked in.

Deleting a file in a later layer doesn't remove it. If a secret ever reached an image, rotate the secret.

Build, run, then look inside

TERMINAL

```
$ TAG=$(git rev-parse --short HEAD)
$ docker build -t ticket-triage:$TAG .
$ docker run -d -p 8080:8080 --name triage ticket-triage:$TAG
$ docker exec triage cat .env
```

`TAG`
your short commit ID: the image is named after its code

`-d · -p 8080:8080`
run in the background · connect your port to the box

`docker exec`
run a command inside the running container

`cat .env`
the token, in plain text, inside the image

YOU SHOULD SEE {"label": "urgent", "confidence": 0.976} from /predict, then your token printed from inside the box

“latest” is not a version

VAGUE

ticket-triage:latest

Points at whatever was built last. Changes silently. Which model is in production? Nobody knows.

VS

TRACEABLE

ticket-triage:a20e20a

The git commit it was built from. Never changes meaning. Image → commit → receipt → data.



Whatever you COPY, everyone who pulls the image can read.

Copy only what runs. Add a .dockerignore. Secrets at run time. Tag with the commit, never just “latest”.

03

MODULE 3 OF 3

No Green, No Ship

A model ships only if a robot says it's good enough.

One commit. Half the accuracy.

92.9%

accuracy on main

1 line

“tune: faster training (1 epoch)”

48.3%

accuracy after the change

0

tests that failed

Unit tests check that code runs. Only a quality gate checks that the model is still good.

Every push walks the same path

1

TEST

unittest: the data and params look right, and the seed is pinned.

2

TRAIN

train.py: reproducible, from Module 1. Writes the receipt.

3

GATE

evaluate.py: exit 0 if accuracy ≥ 85%, exit 1 if not.

4

BUILD

docker build: the safe image from Module 2, tagged with the commit.

Three rules for a fair gate

01

Set the bar first

Decide 85% before the run, not after you see 84.6%. "min_accuracy": 0.85 in git.

02

Same test set

Measure every model on the same held-out data, or the numbers don't compare.

03

More than accuracy

Grown-up gates also check latency, size, fairness and 'no worse than production'.

The bar doesn't move. The change has to clear it.

Watch the gate block a bad model

TERMINAL

```
$ bash pipeline.sh
$ bash .lab/break_it.sh && git log --oneline -1
$ bash pipeline.sh; echo "exit code: $?"
$ git revert --no-edit HEAD && bash pipeline.sh |
tail -3
```

`set -e`
pipeline.sh stops at the first failing command

`break_it.sh`
commits 'tune: faster training (1 epoch)'

`$?`
the last exit code: 0 = success

`git revert`
undo with a new, honest commit, then green

YOU SHOULD SEE GATE FAIL accuracy 48.3% < bar 85% - this model does not ship · exit code: 1

Red is the system working

1

Read the stage

Which box went red? TEST, TRAIN, GATE or BUILD?

2

Find the change

git log --oneline -3 · git show HEAD

3

Revert or fix

git revert HEAD for a quick, safe undo. Then fix properly.

4

Push, go green

The same pipeline proves the fix. No green, no ship.



Your users should never be the ones who find out.

Tests check that code runs. The gate checks the model is still good. Set the bar first, in git. No green, no ship.

Three phrases to keep

Same inputs, same model

Module 1: pin code, data, params, environment. A receipt for every model.

Pack it once. Run it anywhere.

Module 2: a safe container. Nothing secret in the box, not root, tagged by commit.

No green, no ship

Module 3: test, train, gate, build on every push. Red means nothing ships.

YOU CAN DO THIS



Reproducible. Packed safely. Gated on every push. You can do this.

Three modules ago, a model was something that worked on a laptop. Next in the path: SRE for AI-ML Systems.

CODE ALONG

Set up your lab before the next session

1

Pick where to run it

Easiest: GitHub Codespaces. Or Windows, or Mac.

2

Get the lab files

Download devops-for-ai-ml-lab.zip from the repo's downloads folder, then unzip it.

3

Install and check

Follow SETUP.md step by step. The check tells you when you're ready.

4

Run one test per module

SETUP.md has one quick command for each of the 3 modules.

Everything is here: **github.com/Here2ServeU/live-devops-for-ai-ml** · slides, setup and labs

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- **DevOps for AI-ML Systems**
- SRE for AI-ML Systems
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	Linux for AI-ML Systems	Zero to AI-ML Systems Engineering	NEXT SRE for AI-ML Systems	FinOps for AI-ML Systems
THURSDAY	Python for AI-ML	TONIGHT DevOps for AI-ML Systems	GPU Optimization for AI-ML Systems	AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Monday, 7:30 PM Central · SRE for AI-ML Systems

Questions? Let's take them now.

Emmanuel N. · Transformed 2 Succeed